

Artificial Intelligence

Prof. Chang-Hee Han

9

Deep learning for timeseries

9.0 This chapter covers

▣ Today's lecture covers

- Examples of machine learning tasks that involve timeseries data
- Understanding recurrent neural networks (RNNs)
- Applying RNNs to a temperature-forecasting example

10.1 Different kinds of timeseries tasks

▣ Various Types of Time Series Tasks

- **Time series** data means all data measured at regular time intervals.
- Examples include daily stock prices, hourly electricity use in a city, and weekly store sales.
- Time series appear in **natural phenomena** (for example, earthquake activity, changes in fish population in a river, or weather in a region) and in **human activities** (for example, website visitors, national GDP, or credit card transactions).
- Unlike other types of data, time series analysis requires understanding the **system dynamics.**
- It includes **periodicity, time trends, regular patterns, and sudden increases.**

10.1 Different kinds of timeseries tasks

▣ Various Types of Time Series Tasks

- The most common time series task is **forecasting**.
- Forecasting means predicting what will happen next based on current time series data.
- For example, predicting **electricity usage** in a few hours helps estimate demand.
- Predicting **revenue** in a few months helps plan budgets.
- Predicting **weather** in a few days helps plan schedules.
- The focus of this section is **forecasting**.
- However, time series tasks can also be very diverse.
  - **Classification**: Assigning one or more categorical labels to a time series.
  - For example, given a time series of website visitor activity, we can classify whether the visitor is a **bot** or a **human**.

10.1 Different kinds of timeseries tasks

▣ Various Types of Time Series Tasks

- When working with time series, you can find many specialized data representation techniques across different fields.
- For example, you may have heard of the **Fourier Transform**, which separates a time series into different frequency components.
- The Fourier Transform is especially useful for data characterized by **periodicity** and **vibrations**, such as sound, vibrations in tall buildings, or brain waves (EEG).
- In deep learning, **Fourier analysis** (or **Mel-frequency analysis**) and other domain-specific representations are often used as **feature engineering** methods. These techniques preprocess the data before model training, making the learning task easier.
- This section does not cover these feature extraction methods in detail.
- Instead, the focus is on the **modeling part**.

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- This dataset was collected from a weather station at the Max Planck Institute for Biogeochemistry in Jena, Germany.
- It contains 14 different measurements (such as temperature, air pressure, humidity, and wind direction) recorded every 10 minutes over several years.
- The original data have been recorded since 2003, but in this example, only data from 2009 to 2016 are used.
- The first step is to download and prepare the dataset.

```
!wget https://s3.amazonaws.com/keras-datasets/jena_climate_2009_2016.csv.zip
!unzip jena_climate_2009_2016.csv.zip
```

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- Now let's look at the data.

Listing 10.1 Inspecting the data of the Jena weather dataset

```
import os
fname = os.path.join("jena_climate_2009_2016.csv")

with open(fname) as f:
    data = f.read()

lines = data.split("\n")
header = lines[0].split(",")
lines = lines[1:]
print(header)
print(len(lines))
```

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- This outputs a count of 420,551 lines of data (each line is a timestep: a record of a date and 14 weather-related values), as well as the following header:

```
["Date Time",
 "p (mbar)",
 "T (degC)",
 "Tpot (K)",
 "Tdew (degC)",
 "rh (%)",
 "VPmax (mbar)",
 "VPact (mbar)",
 "VPdef (mbar)",
 "sh (g/kg)",
 "H2OC (mmol/mol)",
 "rho (g/m**3)",
 "wv (m/s)",
 "max. wv (m/s)",
 "wd (deg)"]
```

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- Now, convert all 420,551 lines of data into NumPy arrays: one array for the temperature (in degrees Celsius), and another one for the rest of the data — the features we will use to predict future temperatures.
- Note that we discard the “Date Time” column.

Listing 10.2 Parsing the data

```
import numpy as np
temperature = np.zeros((len(lines),))
raw_data = np.zeros((len(lines), len(header) - 1))
for i, line in enumerate(lines):
    values = [float(x) for x in line.split(",")[1:]]
    temperature[i] = values[1]
    raw_data[i, :] = values[:]
```

We store column 1 in the “temperature” array.

We store all columns (including the temperature) in the “raw_data” array.

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- Figure 10.1 shows the plot of temperature (in degrees Celsius) over time.
- On this plot, you can clearly see the yearly periodicity of temperature — the data spans 8 years.

Listing 10.3 Plotting the temperature timeseries

```
from matplotlib import pyplot as plt
plt.plot(range(len(temperature)), temperature)
```

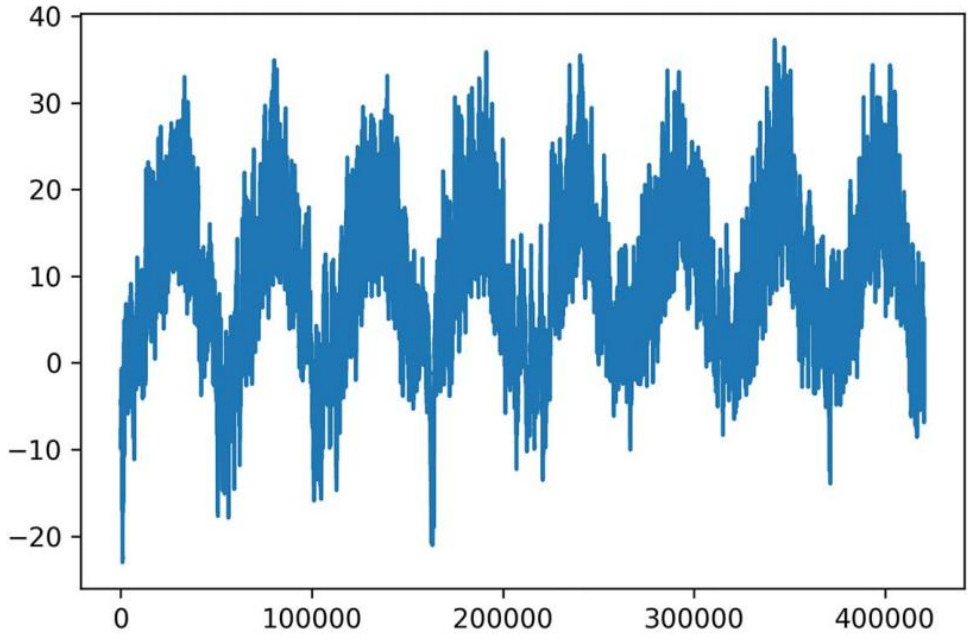


Figure 10.1 Temperature over the full temporal range of the dataset (°C)

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- Figure 10.2 shows a more narrow plot of the first 10 days of temperature data.
- Because the data is recorded every 10 minutes, you get 24 × 6 = 144 data points per day.

Listing 10.4 Plotting the first 10 days of the temperature timeseries

```
plt.plot(range(1440), temperature[:1440])
```

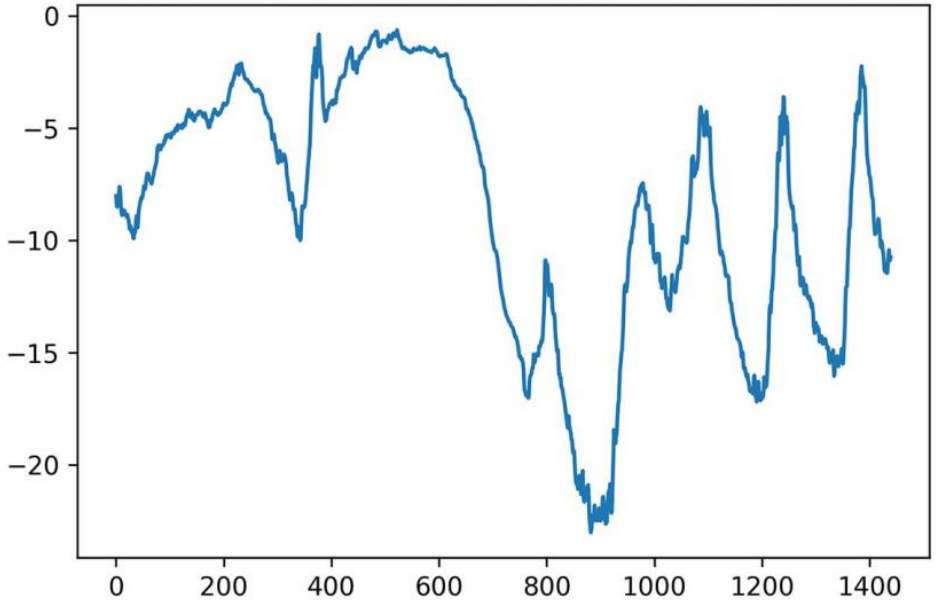


Figure 10.2 Temperature over the first 10 days of the dataset (°C)

- On this plot, you can see daily periodicity, especially for the last 4 days.
- Also note that this 10-day period must be coming from a fairly cold winter month.

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- Always Look for Periodicity in Data
  - ➢ Periodicity over different time ranges is an important and very common property in time series data.
  - ➢ You can find daily and yearly cycles in many types of data such as weather, shopping mall parking spaces, website traffic, grocery store sales, and the number of steps recorded by a fitness tracker.
    - Even data generated by humans often show **weekly periodic patterns**.
  - ➢ When exploring data, always look for these recurring patterns.

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- Using data from the past few months to predict the average temperature of the next month is relatively easy.
- This is because annual periodicity in the data is quite stable.
- However, when looking at day-to-day data, temperature changes are highly unstable and irregular.
- Can we predict daily-level time series data accurately?
- Let’s check it directly.

10.2 A temperature-forecasting example

▣ Temperature Prediction Problem

- In all examples, the first 50% of the data is used for training, the next 25% for validation, and the last 25% for testing.
- When working with time series data, the validation and test sets must always be newer than the training data.
- This is because we predict the future based on the past, not the other way around.
- Therefore, datasets must be split according to this temporal order.
- If the time axis were reversed, some problems would become unrealistically easy to solve!

Listing 10.5 Computing the number of samples we'll use for each data split

```
>>> num_train_samples = int(0.5 * len(raw_data))
>>> num_val_samples = int(0.25 * len(raw_data))
>>> num_test_samples = len(raw_data) - num_train_samples - num_val_samples
>>> print("num_train_samples:", num_train_samples)
>>> print("num_val_samples:", num_val_samples)
>>> print("num_test_samples:", num_test_samples)
num_train_samples: 210225
num_val_samples: 105112
num_test_samples: 105114
```

10.2 A temperature-forecasting example

10.2.1 Preparing the data

- The problem is defined as follows:
  - Given 5 days of data sampled once per hour, can we predict the temperature 24 hours later?
- First, we preprocess the data into a form suitable for feeding into a neural network.
- This step is easy because:
  - The data are already numerical, so no vectorization is needed.
- However, each time series has a different scale (for example, air pressure measured in mbar ≈ 1000, while H_2O concentration measured in mmol/mol ≈ 3).
- Therefore, we normalize each time series independently to make their values smaller and within similar ranges.

10.2 A temperature-forecasting example

10.2.1 Preparing the data

- The first 210,225 timesteps are used as the training data, and the mean and standard deviation are calculated from this range for normalization.

Listing 10.6 Normalizing the data

```
mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean
std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std
```

10.2 A temperature-forecasting example

▣ 10.2.1 Preparing the data

- Now, let's create a Dataset object that returns batches of data from the past 5 days and the target temperature 24 hours later.
- The samples in this dataset contain a lot of overlap (for example, sample N and sample N+1 share most of their timesteps).
- Loading all samples into memory would be very inefficient.
- Instead, we keep only the raw_data and temperature arrays in memory, and generate samples on the fly as needed.
- While you could write a Python generator for this task, Keras provides a convenient built-in utility: timeseries_dataset_from_array(), which simplifies this process.
- Using this function saves effort and memory.
- It can be universally applied to any type of time series forecasting task.

10.2 A temperature-forecasting example

▣ 10.2.1 Preparing the data

- Understanding timeseries_dataset_from_array()
  - To understand how the timeseries_dataset_from_array() function works, let's look at a simple example.
  - When you provide a time series data array (via the data parameter), the function extracts overlapping windows from the original sequence (these windows are called "sequences").
  - For example:
    - If data = [0, 1, 2, 3, 4, 5, 6] and sequence_length = 3,
    - Then the function generates samples like: [0, 1, 2], [1, 2, 3], [2, 3, 4], [3, 4, 5], [4, 5, 6].
  - You can also provide a target array using the targets parameter.
  - The first element in the targets array corresponds to the target of the first sequence generated from the data array.
  - In time series forecasting, the targets array typically matches the data array but with a small time shift to represent future values.

10.2 A temperature-forecasting example

10.2.1 Preparing the data

- Understanding timeseries_dataset_from_array()
  - For instance, with data = [0 1 2 3 4 5 6 …] and sequence_length=3, you could create a dataset to predict the next step in the series by passing targets = [3 4 5 6 …].
  - Let's try it:

```
import numpy as np
from tensorflow import keras
int_sequence = np.arange(10)
dummy_dataset = keras.utils.timeseries_dataset_from_array(
    data=int_sequence[:-3],
    targets=int_sequence[3:],
    sequence_length=3,
    batch_size=2,
)

for inputs, targets in dummy_dataset:
    for i in range(inputs.shape[0]):
        print([int(x) for x in inputs[i]], int(targets[i]))
```

Generate an array of sorted integers from 0 to 9.

The sequences we generate will be sampled from [0 1 2 3 4 5 6].

The target for the sequence that starts at data[N] will be data[N + 3].

The sequences will be 3 steps long.

The sequences will be batched in batches of size 2.

10.2 A temperature-forecasting example

▣ 10.2.1 Preparing the data

- Understanding timeseries_dataset_from_array()
  - ➢ This bit of code prints the following results:

```
[0, 1, 2] 3
[1, 2, 3] 4
[2, 3, 4] 5
[3, 4, 5] 6
[4, 5, 6] 7
```

10.2 A temperature-forecasting example

10.2.1 Preparing the data

- We will create three datasets (training, validation, and test) using the timeseries_dataset_from_array() function.
- The following parameter values are used:
  - sampling_rate = 6
    - One data point is sampled **per hour** (since the data are recorded every 10 minutes).
    - In other words, **one out of every 6 data points** is used.
  - sequence_length = 120
    - Each sequence uses data from the **previous 5 days (120 hours)**.
  - delay = sampling_rate * (sequence_length + 24 - 1)
    - The target for each sequence is the **temperature 24 hours after** the end of that sequence.

10.2 A temperature-forecasting example

10.2.1 Preparing the data

- When creating the training dataset, only the first 50% of the data is used:
  - start_index = 0
  - end_index = num_train_samples
- For the validation dataset, the next 25% of the data is used:
  - start_index = num_train_samples
  - end_index = num_train_samples + num_val_samples
- Finally, for the test dataset, the remaining 25% of the samples are used:
  - start_index = num_train_samples + num_val_samples

10.2 A temperature-forecasting example

▣ 10.2.1 Preparing the data

Listing 10.7 Instantiating datasets for training, validation, and testing

```
sampling_rate = 6
sequence_length = 120
delay = sampling_rate * (sequence_length + 24 - 1)
batch_size = 256

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=0,
    end_index=num_train_samples)

val_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=num_train_samples,
    end_index=num_train_samples + num_val_samples)

test_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=num_train_samples + num_val_samples)
```

10.2 A temperature-forecasting example

▣ 10.2.1 Preparing the data

- Each dataset yields a tuple (samples, targets), where samples is a batch of 256 samples, each containing 120 consecutive hours of input data, and targets is the corresponding array of 256 target temperatures.
- Note that the samples are randomly shuffled, so two consecutive sequences in a batch (like samples[0] and samples[1]) aren't necessarily temporally close.

Listing 10.8 Inspecting the output of one of our datasets

```
>>> for samples, targets in train_dataset:
>>>     print("samples shape:", samples.shape)
>>>     print("targets shape:", targets.shape)
>>>     break
samples shape: (256, 120, 14)
targets shape: (256,)
```

10.2 A temperature-forecasting example

▣ 10.2.2 A common-sense, non-machine learning baseline

- Before solving the temperature prediction problem using a black-box deep learning model, let's try a simple baseline method first.
- This step helps confirm whether the problem is reasonable and serves as a benchmark that any advanced machine learning model should surpass.
- Such baseline approaches are also useful when dealing with new problems that lack well-known solutions.
- A classic example is imbalanced classification:
  - Suppose Class A makes up 90% of the dataset and Class B only 10%.
  - A naïve baseline would simply predict every sample as Class A.
  - This classifier would achieve about 90% accuracy, despite being overly simplistic.
- Therefore, a machine learning model must perform better than this baseline (e.g., >90%) to be considered effective.
- Sometimes, however, surpassing such a baseline can be unexpectedly difficult.

10.2 A temperature-forecasting example

▣ 10.2.2 A common-sense, non-machine learning baseline

- In this case, the time series data are continuous and can be assumed to have daily periodicity (for example, today's temperature is likely similar to tomorrow's).
- Therefore, a common-sense baseline approach is to predict that the temperature 24 hours from now will be the same as the current temperature.
- We will evaluate this baseline using the Mean Absolute Error (MAE) metric, defined as follows.

```
np.mean(np.abs(preds - targets))
```

10.2 A temperature-forecasting example

▣ 10.2.2 A common-sense, non-machine learning baseline

- Here's the evaluation loop.

Listing 10.9 Computing the common-sense baseline MAE

```
def evaluate_naive_method(dataset):
    total_abs_err = 0.
    samples_seen = 0
    for samples, targets in dataset:
        preds = samples[:, -1, 1] * std[1] + mean[1]
        total_abs_err += np.sum(np.abs(preds - targets))
        samples_seen += samples.shape[0]
    return total_abs_err / samples_seen

print(f"Validation MAE: {evaluate_naive_method(val_dataset):.2f}")
print(f"Test MAE: {evaluate_naive_method(test_dataset):.2f}")
```

The temperature feature is at column 1, so samples[:, -1, 1] is the last temperature measurement in the input sequence. Recall that we normalized our features, so to retrieve a temperature in degrees Celsius, we need to un-normalize it by multiplying it by the standard deviation and adding back the mean.

10.2 A temperature-forecasting example

▣ 10.2.2 A common-sense, non-machine learning baseline

- The baseline model achieved a validation MAE of 2.44°C and a test MAE of 2.62°C.
- This means that if we always predict the temperature 24 hours later to be the same as the current temperature, the average difference is about 2.5°C.
- While this performance is not terrible, it's clearly not sufficient — no one would launch a weather forecasting service based on such a simple rule.

10.2 A temperature-forecasting example

▣ 10.2.3 Let's try a basic machine learning model

- We have now established a common-sense baseline before attempting a machine learning model.
- Similarly, before testing complex and computationally expensive models like RNNs, it's a good idea to first build a simpler machine learning model — for example, a small fully connected (dense) network.
- This approach provides a solid reference point for comparison and helps determine whether more complex methods truly offer practical improvements.

10.2 A temperature-forecasting example

▣ 10.2.3 Let's try a basic machine learning model

- The following code (Example 10-10) shows a fully connected (Dense) network that flattens the input data and passes it through two Dense layers.
- Because this is a regression problem, no activation function is applied in the final Dense layer.
- The Mean Squared Error (MSE) is used as the loss function instead of MAE.
- MSE is suitable for gradient-based optimization because it is differentiable everywhere.
- In the compile() method, the MAE metric is added to monitor model performance during training.

10.2 A temperature-forecasting example

▣ 10.2.3 Let's try a basic machine learning model

Listing 10.10 Training and evaluating a densely connected model

```
from tensorflow import keras
from tensorflow.keras import layers

inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Flatten()(inputs)
x = layers.Dense(16, activation="relu")(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_dense.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_dense.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

We use a callback to save the best-performing model.

Reload the best model and evaluate it on the test data.

10.2 A temperature-forecasting example

▣ 10.2.3 Let's try a basic machine learning model

- Let's display the loss curves for validation and training (see figure 10.3).

Listing 10.11 Plotting results

```
import matplotlib.pyplot as plt
loss = history.history["mae"]
val_loss = history.history["val_mae"]
epochs = range(1, len(loss) + 1)
plt.figure()
plt.plot(epochs, loss, "bo", label="Training MAE")
plt.plot(epochs, val_loss, "b", label="Validation MAE")
plt.title("Training and validation MAE")
plt.legend()
plt.show()
```

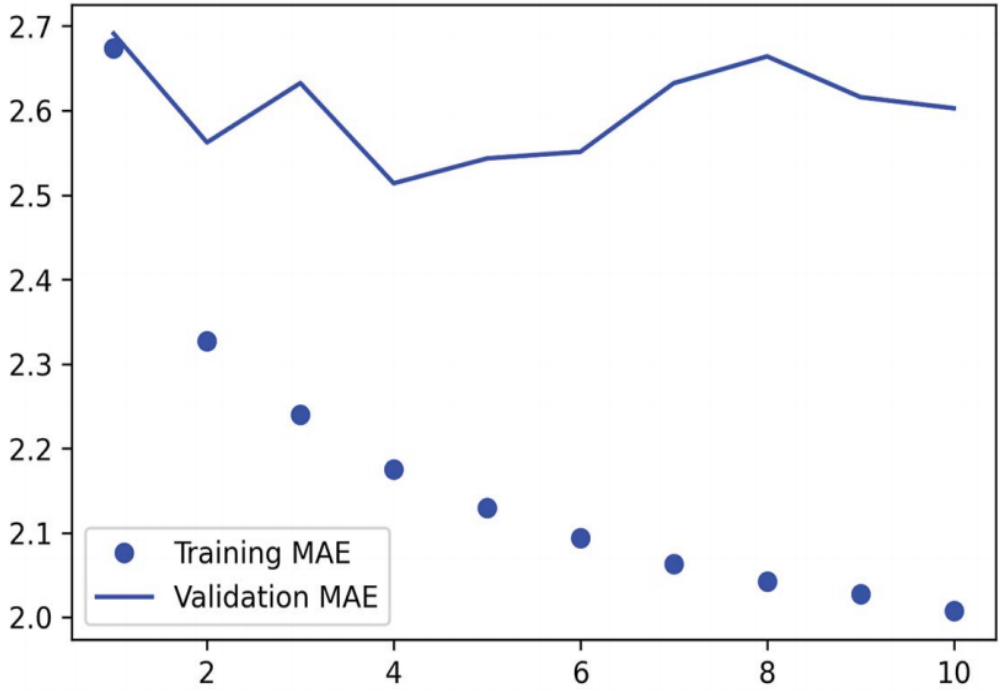


Figure 10.3 Training and validation MAE on the Jena temperature-forecasting task with a simple, densely connected network

10.2 A temperature-forecasting example

▣ 10.2.3 Let's try a basic machine learning model

- Some validation losses are close to the baseline performance, but the model's results are unstable.
- Creating the baseline model earlier proves helpful for comparison and evaluation.
- It turns out that surpassing the baseline in this temperature prediction task is not easy.
- The common-sense assumptions we used (e.g., "tomorrow's temperature ≈ today's temperature") actually contain a lot of core information that the machine learning model struggles to capture directly.

10.2 A temperature-forecasting example

▣ 10.2.3 Let's try a basic machine learning model

- If a simple, reasonable model (like the baseline) can map data to targets effectively, why can't a trained neural network find the same mapping and achieve similar performance?
- The answer lies in the hypothesis space — the set of all possible weight combinations of the 2-layer network we defined.
- The baseline model represents just one configuration within this enormous space.
- Finding it is like searching for a needle in a haystack.
- The existence of a good solution in the hypothesis space doesn't mean that gradient descent will necessarily find it.
- This is one of the fundamental limitations of machine learning.
- Unless the learning algorithm is explicitly guided or hard-coded to find a particular kind of simple model, it might miss even an easy solution.
- This is why good feature engineering and task-aware model architectures are crucial — they help the model focus on what it should learn.

10.2 A temperature-forecasting example

▣ 10.2.4 Let's try a 1D convolutional model

- Since the input sequence has a daily periodic pattern, it makes sense to apply a convolutional model with an appropriate architecture.
- A convolution along the time axis allows the model to reuse patterns that occur on different days.
- This is similar to how spatial convolutions in images reuse the same features at different locations.
- You've already learned about Conv2D and SeparableConv2D layers.
- These layers scan input data across a 2D grid using small windows (kernels).
- Similarly, there are 1D and even 3D convolutional layers available:
  - Conv1D, SeparableConv1D, and Conv3D.
- A Conv1D layer slides 1D windows across the input sequence.
- A Conv3D layer slides 3D cubic windows across input volumes.

10.2 A temperature-forecasting example

▣ 10.2.4 Let's try a 1D convolutional model

- You can build a 1D convolutional network (1D ConvNet) that is very similar to a 2D ConvNet.
- This approach works well for sequence data that follow the translation invariance assumption — meaning the same local pattern should be recognized regardless of its position in the sequence.
- Let's apply this concept to the temperature prediction problem.
- The initial window length is set to 24, so the model observes one full daily cycle (24 hours) at a time.
- Because the sequence will later be downsampled using a MaxPooling1D layer, the window size is reduced accordingly to keep features aligned.

10.2 A temperature-forecasting example

▣ 10.2.4 Let's try a 1D convolutional model

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Conv1D(8, 24, activation="relu")(inputs)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 12, activation="relu")(x)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(8, 6, activation="relu")(x)
x = layers.GlobalAveragePooling1D()(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_conv.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_conv.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

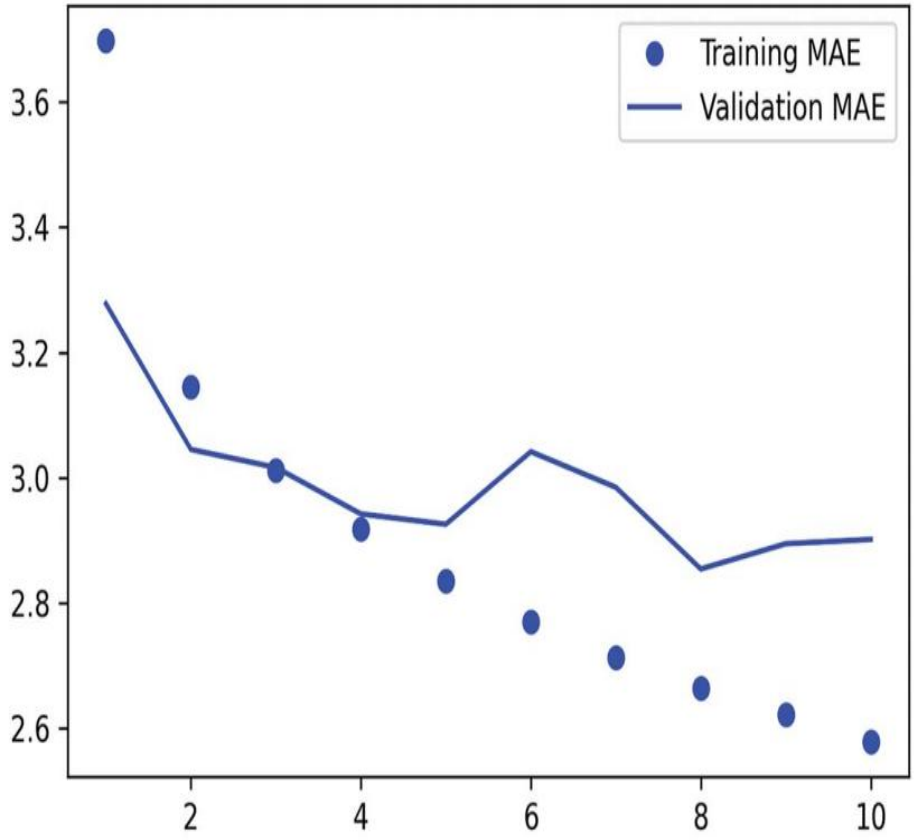


Figure 10.4 Training and validation MAE on the Jena temperature-forecasting task with a 1D convnet

10.2 A temperature-forecasting example

▣ 10.2.4 Let's try a 1D convolutional model

- The performance of this model is worse than that of the fully connected model.
- It achieved a validation MAE of about 2.9°C, showing a large gap compared to the baseline model.
- What's the problem?
- There are two main issues:
  - ➢ (1) Weather data do not strongly follow the translation invariance assumption.
    - Although the data show daily periodicity, morning data differ in nature from evening or midnight data.
    - Weather data have translation invariance only within specific time ranges.
  - ➢ (2) The order of data matters greatly.
    - Recent data are much more useful than data from five days ago for predicting tomorrow's temperature.
    - A 1D ConvNet cannot take advantage of this property.
    - In particular, layers like MaxPooling and GlobalAveragePooling remove much of the sequential information.

10.2 A temperature-forecasting example

▣ 10.2.5 A first recurrent baseline

- Although the dense and convolutional models didn't perform well, this doesn't mean that machine learning is unsuitable for this problem.
- The dense model lost the concept of time because it flattened the time series data.
- The convolutional model processed all parts of the data similarly and, through pooling, lost much of the sequential information.
- Instead, we will now use sequence data as they are — preserving causal relationships and temporal order — to see how they perform.

10.2 A temperature-forecasting example

10.2.5 A first recurrent baseline

- **A Recurrent Neural Network (RNN)** is a neural architecture specifically designed for problems like this.
- Among RNN types, the **LSTM (Long Short-Term Memory)** layer has long been one of the most popular.
- Let's now try applying an **LSTM layer** to this task.

Listing 10.12 A simple LSTM-based model

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)
model = keras.models.load_model("jena_lstm.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

10.2 A temperature-forecasting example

10.2.5 A first recurrent baseline

- **A Recurrent Neural Network (RNN)** is a neural architecture specifically designed for problems like this.
- Among RNN types, the **LSTM (Long Short-Term Memory)** layer has long been one of the most popular.
- Let's now try applying an **LSTM layer** to this task.

Listing 10.12 A simple LSTM-based model

```
inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)
model = keras.models.load_model("jena_lstm.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

10.2 A temperature-forecasting example

▣ 10.2.5 A first recurrent baseline

- The lowest validation MAE was 2.36°C, and the test MAE was 2.55°C.
- The LSTM-based model finally outperformed the baseline model (though only slightly), demonstrating the value of machine learning for this task.

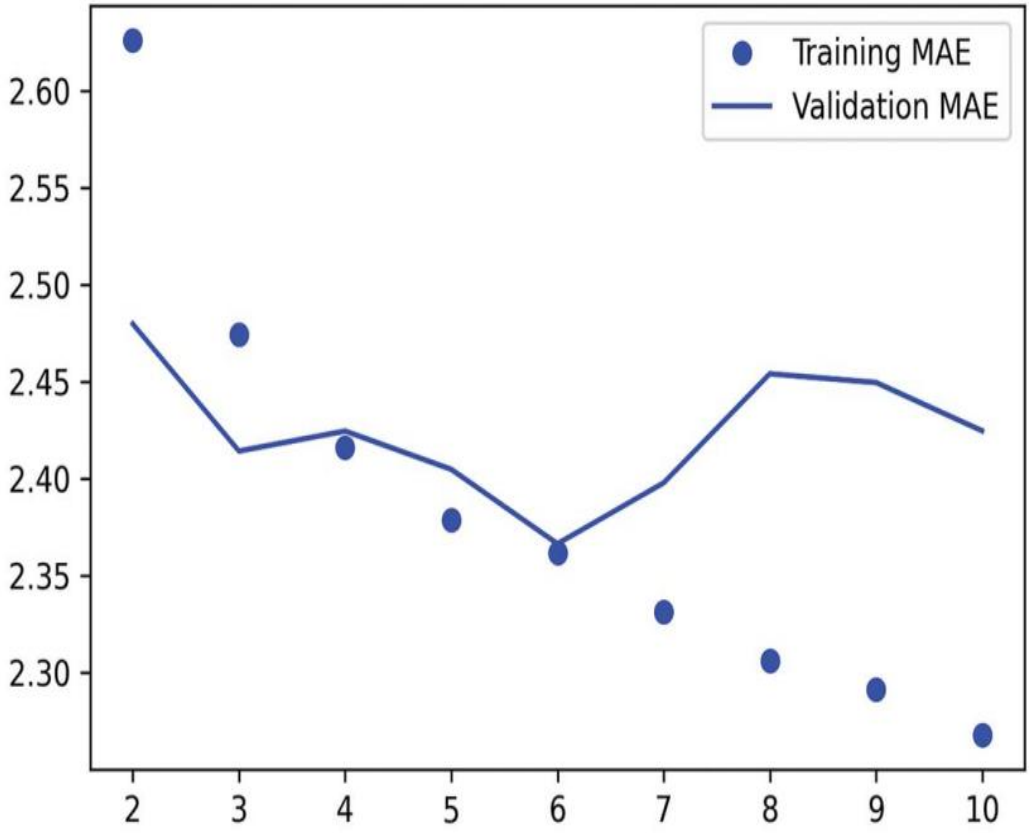


Figure 10.5 Training and validation MAE on the Jena temperature-forecasting task with an LSTM-based model (note that we omit epoch 1 on this graph, because the high training MAE (7.75) at epoch 1 would distort the scale)

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- Unlike dense or convolutional networks, all the models we've seen so far lack memory.
- The input given to the network is processed independently, with no state maintained between inputs.
- To process sequential or time-series data using such networks, the entire sequence must be provided to the network at once.
- In other words, the whole sequence must be flattened into a single data point.
- For example, as seen in the dense model, 5 days of data were flattened into one large vector.
- These types of networks are called feedforward networks.

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- In contrast, when humans read a sentence, they remember what came before while processing each new word.
- This helps us naturally understand and represent the meaning of the whole sentence.
- Similarly, biological intelligence maintains an internal model and processes information gradually over time.
- Such models use past information to build understanding and continuously update with new input.
- A Recurrent Neural Network (RNN) is a simplified version of this principle.
- It processes each element in a sequence step by step, storing previous information in a state.
- In essence, an RNN is a neural network with internal loops, allowing it to retain memory over time.

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

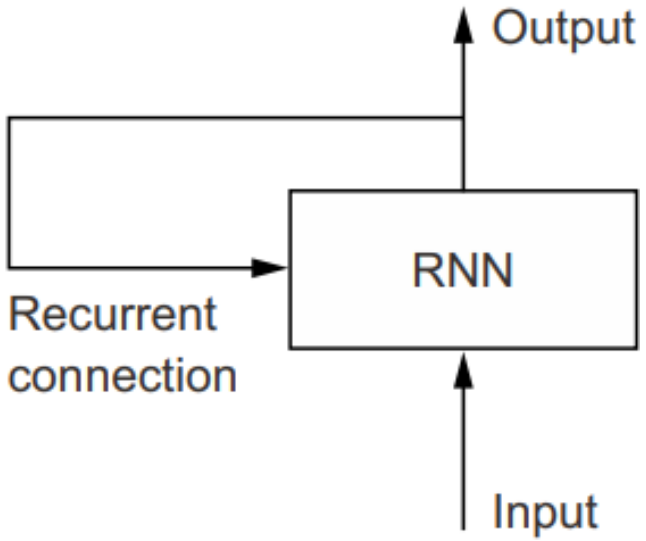


Figure 10.6 A recurrent network: a network with a loop

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- The state of an RNN is reset between processing different sequences (e.g., between two samples in a batch).
- A single sequence is still treated as one data point — one input fed into the network.
- The difference is that this input is not processed all at once.
- Instead, the network iterates internally over the elements of the sequence, step by step.

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- To clarify the concept of loops and states, let's look at a simple RNN forward-pass computation.
- The RNN receives a sequence of vectors encoded as a rank-2 tensor with shape (timesteps, input_features).
- As it loops through each timestep t, it computes an output by combining the current input (of shape (input_features,)) with the current state.
- This output is then used as the state for the next timestep.
- At the first timestep, there is no previous output, so no current state exists yet.
- In that case, the network initializes its state as a zero vector (initial state).

Understanding recurrent neural networks

- In pseudocode, this is the RNN.

Listing 10.13 Pseudocode RNN

```
state_t = 0                          # The state at t
for input_t in input_sequence:       # Iterates over sequence elements
    output_t = f(input_t, state_t)
    state_t = output_t               # The previous output becomes the state for the next iteration.
```

Understanding recurrent neural networks

- The function f transforms the input and the previous state into an output.
- This transformation can be expressed using two matrices, W and U, along with a bias vector.
- The process is similar to the transformation performed in a Dense (fully connected) layer of a feedforward network.

Listing 10.14 More-detailed pseudocode for the RNN

```
state_t = 0
for input_t in input_sequence:
    output_t = activation(dot(W, input_t) + dot(U, state_t) + b)
    state_t = output_t
```

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- To make these notions absolutely unambiguous, let's write a naive NumPy implementation of the forward pass of the simple RNN.

Listing 10.15 NumPy implementation of a simple RNN

```
import numpy as np
timesteps = 100
input_features = 32
output_features = 64
inputs = np.random.random((timesteps, input_features))
state_t = np.zeros((output_features,))
W = np.random.random((output_features, input_features))
U = np.random.random((output_features, output_features))
b = np.random.random((output_features,))
successive_outputs = []
for input_t in inputs:
    output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
    successive_outputs.append(output_t)
    state_t = output_t
final_output_sequence = np.stack(successive_outputs, axis=0)
```

Input data: random noise for the sake of the example

Number of timesteps in the input sequence

Dimensionality of the input feature space

Dimensionality of the output feature space

Initial state: an all-zero vector

Creates random weight matrices

Stores this output in a list

Combines the input with the current state (the previous output) to obtain the current output. We use tanh to add non-linearity (we could use any other activation function).

The final output is a rank-2 tensor of shape (timesteps, output_features).

input_t is a vector of shape (input_features,).

Updates the state of the network for the next timestep

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- In summary, an RNN is essentially a for-loop that reuses previously computed information during each iteration.
- There are, of course, many variations of RNNs that follow this basic definition.
- The example here represents the simplest form of an RNN.
- An RNN is characterized by its step function, which defines how the state is updated at each timestep.
- In this example, the step function operates as shown in Figure 10-7.

```
output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
```

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

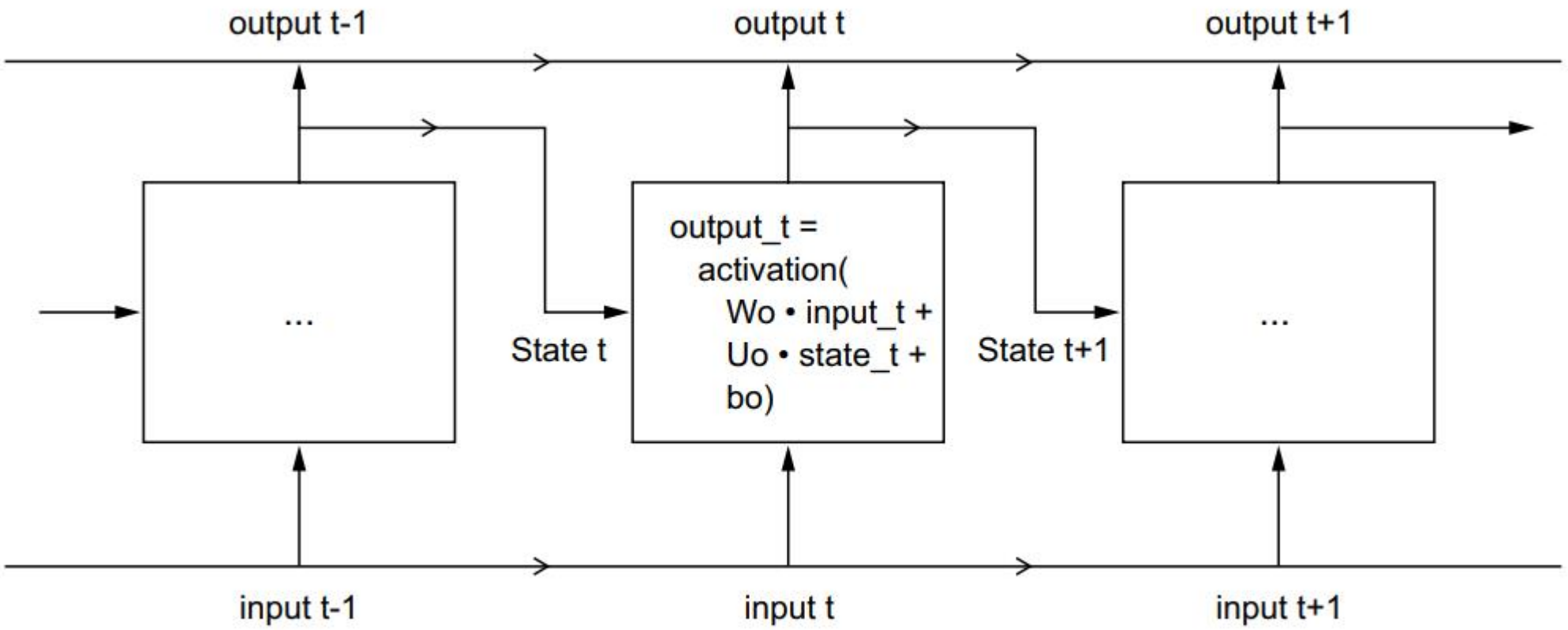


Figure 10.7 A simple RNN, unrolled over time

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- In summary, an RNN is essentially a for-loop that reuses previously computed information during each iteration.
- There are, of course, many variations of RNNs that follow this basic definition.
- The example here represents the simplest form of an RNN.
- An RNN is characterized by its step function, which defines how the state is updated at each timestep.
- In this example, the step function operates as shown in Figure 10-7.

```
output_t = np.tanh(np.dot(W, input_t) + np.dot(U, state_t) + b)
```

10.3 Understanding recurrent neural networks

▣ Understanding recurrent neural networks

- In this example, the final output is a rank-2 tensor with shape (timesteps, output_features).
- Each timestep represents the output at time *t*.
- The output tensor at timestep *t* contains information about all previous inputs from timestep 0 up to *t*.
- Therefore, in many cases, the entire output sequence is unnecessary.
- Since the final output already holds information about the entire sequence, only the last output (output_t) from the loop is typically used.

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- The simple NumPy-based implementation corresponds to Keras's SimpleRNN layer.
- One difference is that, unlike the NumPy example which processes a single sequence, SimpleRNN in Keras handles batches of sequences, just like other Keras layers.
- Thus, it expects inputs with shape (batch_size, timesteps, input_features) instead of (timesteps, input_features).
- When defining the input shape using the Input() function, you can set the timesteps dimension to None.
- This allows the model to process sequences of variable lengths.

Listing 10.16 An RNN layer that can process sequences of any length

```
num_features = 14
inputs = keras.Input(shape=(None, num_features))
outputs = layers.SimpleRNN(16)(inputs)
```

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- This approach is especially useful when the model needs to process variable-length sequences.
- However, if all sequences have the same length, it's better to specify the full input shape explicitly.
- Doing so allows model.summary() to display detailed output shape information, which is always helpful and can also enable certain performance optimizations.

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- All recurrent layers in Keras — SimpleRNN, LSTM, and GRU — can operate in two modes.
- They can either:
  - Return the **entire sequence of outputs** for all timesteps → a rank-3 tensor with shape **(batch_size, timesteps, output_features)**, or
  - Return **only the last output** of the input sequence → a rank-2 tensor with shape **(batch_size, output_features)**.
- These two modes are controlled by the return_sequences argument in the layer constructor.
- Let's look at an example using SimpleRNN, where only the final timestep's output is returned.

Listing 10.17 An RNN layer that returns only its last output step

```
>>> num_features = 14
>>> steps = 120
>>> inputs = keras.Input(shape=(steps, num_features))
>>> outputs = layers.SimpleRNN(16, return_sequences=False)(inputs)
>>> print(outputs.shape)
(None, 16)
```

Note that return_sequences=False is the default.

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- The following example returns the full state sequence.

Listing 10.18 An RNN layer that returns its full output sequence

```
>>> num_features = 14
>>> steps = 120
>>> inputs = keras.Input(shape=(steps, num_features))
>>> outputs = layers.SimpleRNN(16, return_sequences=True)(inputs)
>>> print(outputs.shape)
(120, 16)
```

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- To increase the representational power of the network, it can be useful to stack multiple recurrent layers sequentially.
- In such configurations, the intermediate layers should be set to return the full output sequence, allowing the next recurrent layer to process the entire sequence as its input.

Listing 10.19 Stacking RNN layers

```
inputs = keras.Input(shape=(steps, num_features))
x = layers.SimpleRNN(16, return_sequences=True)(inputs)
x = layers.SimpleRNN(16, return_sequences=True)(x)
outputs = layers.SimpleRNN(16)(x)
```

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- In practice, the SimpleRNN layer is rarely used.
- It is generally too simple for real-world applications.
- Theoretically, SimpleRNN can retain information from all previous timesteps up to the current time *t*.
- However, in practice, it cannot learn long-term dependencies.
- This is due to the vanishing gradient problem, a phenomenon similar to what occurs in deep feedforward networks when many layers are added.
- As the network deepens, training becomes increasingly difficult.
- In the early 1990s, Hochreiter, Schmidhuber, and Bengio studied the theoretical causes of this problem.

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- Fortunately, SimpleRNN is not the only recurrent layer available in Keras.
- To address its limitations, more advanced layers such as LSTM and GRU were developed.
- Let's take a closer look at the LSTM layer.
- The Long Short-Term Memory (LSTM) algorithm was developed by Hochreiter and Schmidhuber in 1997.
- It represents a major breakthrough in solving the vanishing gradient problem in recurrent networks.

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- The LSTM layer is a variant of SimpleRNN, with an added mechanism that allows information to be carried across multiple timesteps.
- Imagine a conveyor belt running alongside the sequence being processed.
- Information extracted at one point in the sequence can be placed on the conveyor belt and moved forward to later timesteps where it is needed.
- This is essentially what LSTM does — it stores information for future use, preventing older signals from fading away during long sequences.
- This idea is similar to the residual connections introduced in Chapter 9.
- In fact, the two concepts are closely related.

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- To understand this in more detail, let's start by examining the structure of a SimpleRNN cell (see Figure 10-8).
- Multiple weight matrices are involved in this computation, so the matrices related to the output are denoted as W_o and U_o, representing the W and U matrices inside the cell that correspond to the output (o).

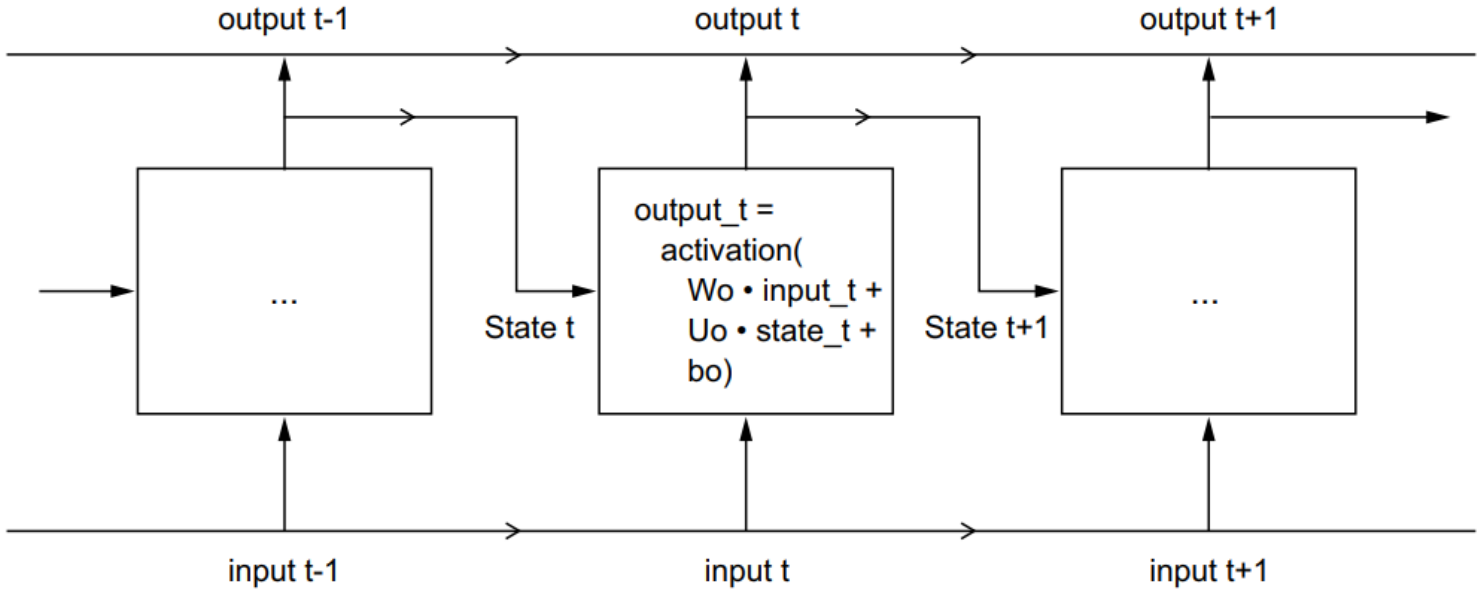


Figure 10.8 The starting point of an `LSTM` layer: a `SimpleRNN`

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- In Figure 10-8, let's add a data flow that carries information across timesteps.
- At timestep n, this value is called the carry state, denoted as c_t.
- Here, c stands for carry.
- Using this information, the cell is modified as follows:
- The carry state is connected to both the input connection and the recurrent connection (state).
- After being multiplied by weight matrices, added with bias, and passed through an activation function, it behaves similarly to a dense layer transformation.
- This state then influences the next timestep's state, through activation and multiplication operations.
- Conceptually, this data-carrying flow regulates the next output and state (see Figure 10-9).
- Up to this point, the mechanism remains fairly simple.

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

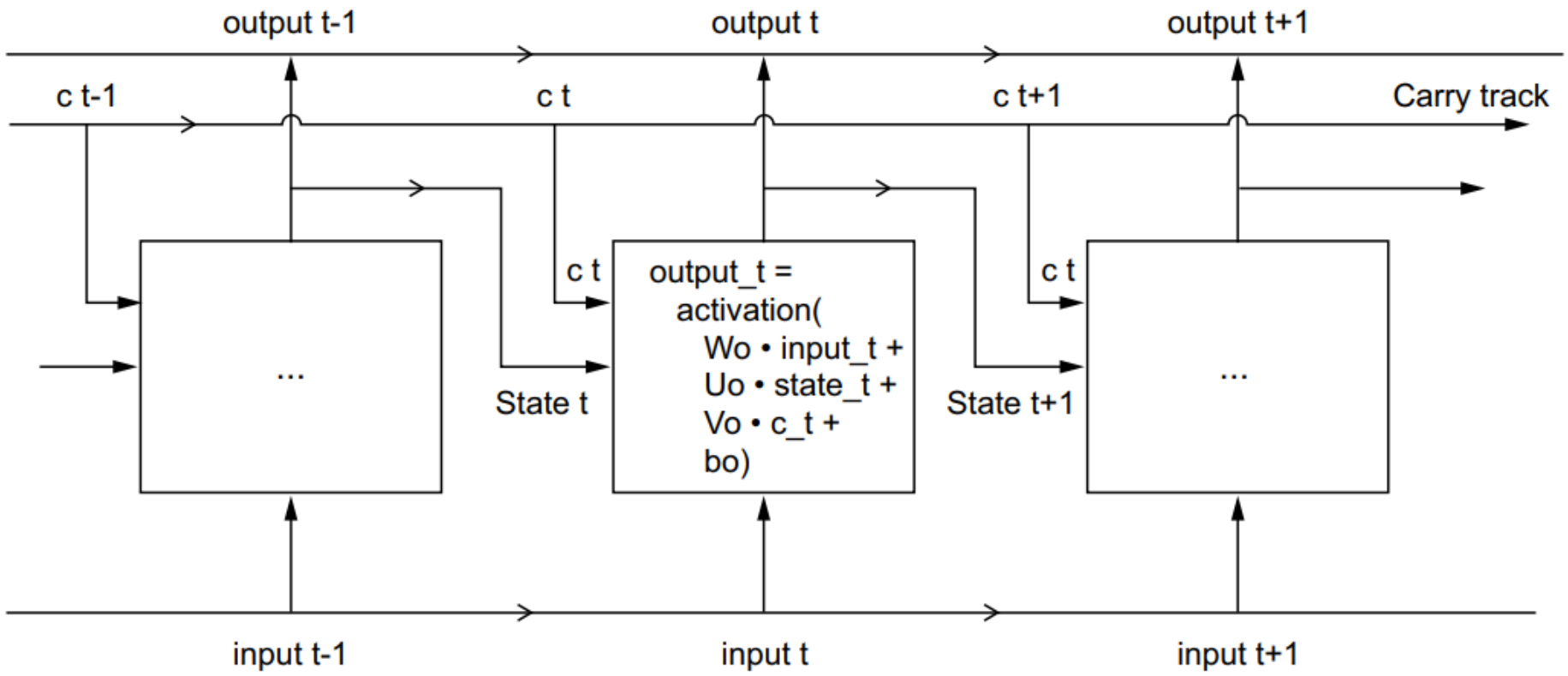


Figure 10.9 Going from a SimpleRNN to an LSTM: adding a carry track

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- The complex part of the data flow lies in how the next carry state (c_{t+1}) is calculated.
- This process involves three different transformations.
- All three have the same structure as those in a SimpleRNN cell.

```
y = activation(dot(state_t, U) + dot(input_t, W) + b)
```

- Each transformation has its own weight matrices.
- These transformations are denoted as i, f, and k.

Listing 10.20 Pseudocode details of the LSTM architecture (1/2)

```
output_t = activation(dot(state_t, Uo) + dot(input_t, Wo) + dot(c_t, Vo) + bo)
i_t = activation(dot(state_t, Ui) + dot(input_t, Wi) + bi)
f_t = activation(dot(state_t, Uf) + dot(input_t, Wf) + bf)
k_t = activation(dot(state_t, Uk) + dot(input_t, Wk) + bk)
```

10.3 Understanding recurrent neural networks

10.3.1 A recurrent layer in Keras

- We obtain the new carry state (the next c_t) by combining i_t, f_t, and k_t.

Listing 10.21 Pseudocode details of the LSTM architecture (2/2)

```
c_t+1 = i_t * k_t + c_t * f_t
```

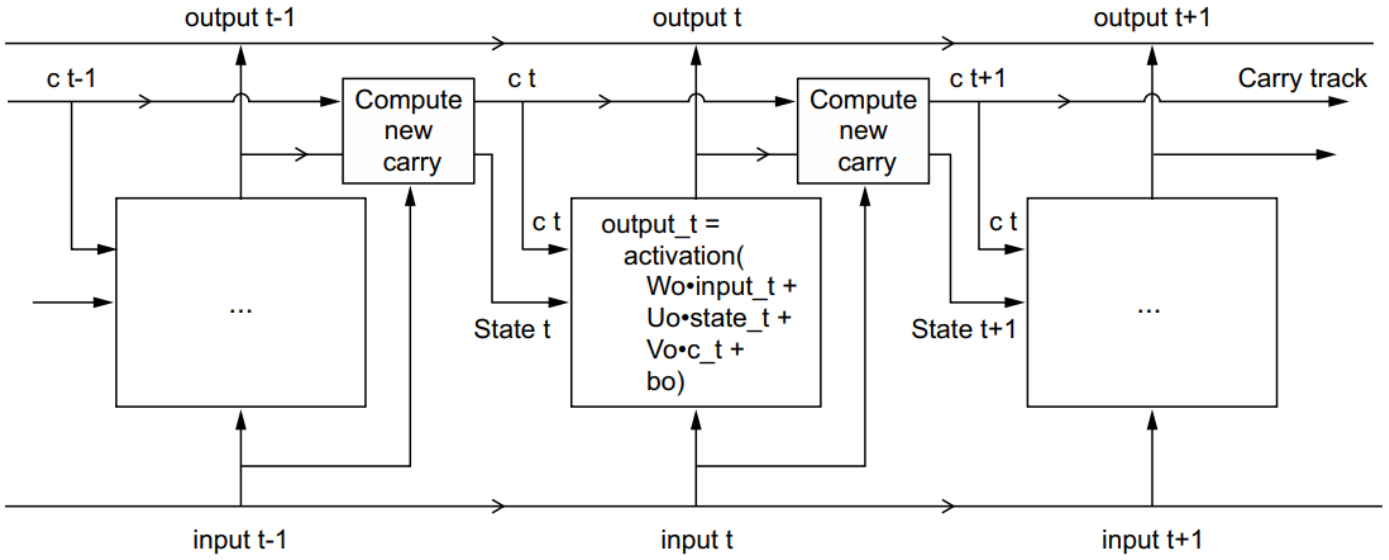


Figure 10.10 Anatomy of an LSTM

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- By analyzing these operations, we can gain some intuition about their roles.
- For example, the multiplication of c_t and f_t can be seen as intentionally removing less relevant information from the data flow.
- Meanwhile, i_t and k_t provide new information about the current input and update the carry track with fresh data.
- However, such interpretations are mostly heuristic and not mathematically essential.

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- The behavior of these operations is entirely determined by their associated weight matrices.
- These weights are learned end-to-end during training.
- Since training restarts fresh each time, it′s impossible to assign fixed, explicit purposes to individual operations.
- The structure of an RNN cell defines the hypothesis space — the set of possible functions the model can represent.
- During training, the network searches this space for optimal parameters.
- Therefore, the cell′s structure does not dictate what it specifically does — its weights do.
- Even with identical cell architectures, different weights can lead to very different behaviors.
- Thus, the composition of operations within an RNN cell should be viewed not as manual engineering, but as a set of constraints defining the hypothesis space the model can explore.

10.3 Understanding recurrent neural networks

▣ 10.3.1 A recurrent layer in Keras

- Designing RNN cell constraints (like LSTM structure) is better handled by optimization algorithms such as genetic algorithms or reinforcement learning, rather than manual engineering.
- Future networks will likely be built this way.
- Therefore, there's no need to deeply understand the specific internal structure of an LSTM cell.
- What matters is its role, not its mechanics.
- The key idea: LSTM stores past information and reinjects it later to overcome the vanishing gradient problem.



Q & A

Prof. Chang-Hee Han